DevOps

DevOps is the integration and automation of software development and information technology operations.[a] DevOps encompasses necessary tasks of software development and can lead to shortening development time and improving the development life cycle.[1] According to Neal Ford, DevOps, particularly through continuous delivery, employs the "Bring the pain forward" principle, tackling tough tasks early, fostering automation and swift issue detection.[2] Software programmers and architects should use fitness functions to keep their software in check.[3]

Although debated,[b][c][d][e] DevOps is characterized by key principles: shared ownership, workflow automation, and rapid feedback. From an academic perspective, Len Bass, Ingo Weber, and Liming Zhu—three computer science researchers from the CSIRO and the Software Engineering Institute—suggested defining DevOps as "a set of practices intended to reduce the time between committing a change to a system and the change being placed into normal production, while ensuring high quality".[7] However, the term is used in multiple contexts. At its most successful, DevOps is a combination of specific practices, culture change, and tools.[8]

History

Proposals to combine software development methodologies with deployment and operations concepts began to appear in the late 80s and early 90s.[9]

In 2009, the first conference named DevOps Days was held in Ghent, Belgium. The conference was founded by Belgian consultant, project manager and agile practitioner Patrick Debois.[10][11] The conference has now spread to other countries.[12]

In 2012, a report called "State of DevOps" was first published by Alanna Brown at Puppet Labs.[13][14]

As of 2014, the annual State of DevOps report was published by Nicole Forsgren, Gene Kim, Jez Humble and others. They stated that the adoption of DevOps was accelerating.[15][16] Also in 2014, Lisa Crispin and Janet Gregory wrote the book More Agile Testing, containing a chapter on testing and DevOps.[17][18]

In 2016, the DORA metrics for throughput (deployment frequency, lead time for changes), and stability (mean time to recover, change failure rate) were published in the State of DevOps report.[13] However, the research methodology and metrics were criticized by experts.[19][20][21][22] In response to these criticisms, the 2023 State of DevOps report [23] published changes that updated the stability metric "mean time to recover" to "failed deployment recovery time" acknowledging the confusion the former metric has caused.[24]

Relevant metrics

DevOps Research and Assessment (DORA) has developed a series of metrics which are intended to measure software development efficiency and reliability. These metrics include:[25][26]

- Deployment Frequency: Time between code deployments.
- Mean Lead Time for Changes: Time between code commit and deployment.
- Change Failure Rate: Percentage of deployments causing production issues.
- Failed Deployment Recovery Time (formerly Mean Time To Recover)
- Reliability (added in 2021): Measures operational performance, focusing on availability and adherence to user expectations.

Relationship to other approaches

Many of the ideas fundamental to DevOps practices are inspired by, or mirror, other well known practices such as Lean and Deming's Plan-Do-Check-Act cycle, through to The Toyota Way and the Agile approach of breaking down components and batch sizes.[27] Contrary to the "top-down" prescriptive approach and rigid framework of ITIL in the 1990s, DevOps is "bottom-up" and flexible, having been created by software engineers for their own needs.[28]

Platform engineering

Platform engineering is an emerging discipline within software engineering that supports DevOps by building and maintaining internal developer platforms (IDPs). These platforms provide standardized tools and reusable components—such as CI/CD pipelines, infrastructure provisioning, observability, and security controls—to streamline software delivery and reduce the cognitive load on developers. The goal is to enable self-service capabilities, improve productivity, and ensure consistency across development and operations teams.[29][30]

Agile

The motivations for what has become modern DevOps and several standard DevOps practices such as automated build and test, continuous integration, and continuous delivery originated in the Agile world, which dates (informally) to the 1990s, and formally to 2001. Agile development teams using methods such as extreme programming couldn't "satisfy the customer through early and continuous delivery of valuable software"[31] unless they took responsibility for operations and infrastructure for their applications, automating much of that work. Because Scrum emerged as the dominant Agile framework in the early 2000s and it omitted the engineering practices that were part of many Agile teams, the movement to automate operations and infrastructure functions splintered from Agile and expanded into what has become modern DevOps. Today, DevOps focuses on the deployment of developed software, whether it is developed using Agile oriented methodologies or other methodologies.

ArchOps

ArchOps presents an extension for DevOps practice, starting from software architecture artifacts, instead of source code, for operation deployment.[32] ArchOps states that architectural models are first-class entities in software development, deployment, and operations.

Continuous Integration and Delivery (CI/CD)

Automation is a core principle for achieving DevOps success and CI/CD is a critical component.[33] Plus, improved collaboration and communication between and within teams helps achieve faster time to market, with reduced risks.[34]

Database DevOps

Database DevOps applies DevOps and CI/CD principles directly to database development and operations. Integrating schema changes, migrations, reference data, and other data-layer updates into the same version-controlled and automated pipelines used for application code enables more reliable deployments and better coordination between application and data changes.[35][36]

Typical practices documented in both research and industry benchmarks include placing database schema definitions under version control, applying automated tests (such as unit tests or migration validation) to database changes, and deploying those changes through CI/CD pipelines. These practices reduce "schema drift" between development and production systems and lower the risk of deployment failures.[37][38]

Mobile DevOps

Mobile DevOps is a set of practices that applies the principles of DevOps specifically to the development of mobile applications. Traditional DevOps focuses on streamlining the software development process in general, but mobile development has its own unique challenges that require a tailored approach.[39] Mobile DevOps is not simply as a branch of DevOps specific to mobile app development, instead an extension and reinterpretation of the DevOps philosophy due to very specific requirements of the mobile world.

Site-reliability engineering

In 2003, Google developed site reliability engineering (SRE), an approach for releasing new features continuously into large-scale high-availability systems while maintaining high-quality end-user experience.[40] While SRE predates the development of DevOps, they are generally viewed as being related to each other. Some of the original authors of the discipline consider SRE as an implementation of DevOps.[41]

Toyota production system, lean thinking, kaizen

Toyota production system, also known under the acronym TPS, was the inspiration for lean thinking with its focus on continuous improvement, kaizen, flow and small batches. The andon cord principle to create fast feedback, swarm and solve problems stems from TPS.[42][43]

DevSecOps, shifting security left

DevSecOps is an augmentation of DevOps to allow for security practices to be integrated into the DevOps approach. Contrary to a traditional centralized security team model, each delivery team is empowered to factor in the correct security controls into their software delivery. Security practices and testing are performed earlier in the development lifecycle, hence the term "shift left". Security is tested in three main areas: static, software composition, and dynamic.

Checking software statically via static application security testing (SAST) is white-box testing with special focus on security. Depending on the programming language, different tools are needed to do such static code analysis. The software composition is analyzed, especially libraries, and the version of each component is checked against vulnerability lists published by CERT and other expert groups. When giving software to clients, library licenses and their match to the license of the software distributed are in focus, especially copyleft licenses.

In dynamic testing, also called black-box testing, software is tested without knowing its inner functions. In DevSecOps this practice may be referred to as dynamic application security testing (DAST) or penetration testing. The goal is early detection of defects including cross-site scripting and SQL injection vulnerabilities.

Often, detected defects from static and dynamic testing are triaged and categorized under taxonomies like the Common Weakness Enumeration (CWE), maintained by the Mitre Corporation. This facilitates the prioritization of security bug fixes and also allows frequently recurring weaknesses to be fixed with recommended mitigations. As of 2025, CWE maintained its own list of frequently-occurring weaknesses, the CWE Top 25.[44] In addition, organizations like Open Worldwide Application Security Project (OWASP) maintain lists of industry-wide frequently recurring software weaknesses.[45]

DevSecOps has also been described as a cultural shift involving a holistic approach to producing secure software by integrating security education, security by design, and security automation.[46]

Culture

DevOps initiatives can change how a company's operations, developers, and testers collaborate during the development and delivery processes.[47][48]

DevOps attempts to support consistency, reliability, and efficiency within an organization. This is usually enabled by a shared code repository or version control.[49] Many organizations use version control to facilitate DevOps automation technologies like virtual machines, containerization (or OS-level virtualization), and CI/CD,[50] with the Git version control system and the GitHub platform referenced as examples.

GitOps

GitOps evolved from DevOps. The specific state of deployment configuration is version-controlled. Because the most popular version-control is Git, the GitOps approach has been named after Git. Changes to configuration can be managed using code review practices, and can be rolled back using version-

controlling. Essentially, all of the changes to a code are tracked, bookmarked, and making any updates to the history can be made easier. As explained by Red Hat, "visibility to change means the ability to trace and reproduce issues quickly, improving overall security."[51]

Best practices for cloud systems

The following practices can enhance productivity of DevOps pipelines, especially in systems hosted in the cloud:[52][53][54]

- **Number of Pipelines**: Small teams can be more productive by having one repository and one pipeline. In contrast, larger organizations may have separate repositories and pipelines for each team or even separate repositories and pipelines for each service within a team.
- **Permissions**: In the context of pipeline-related permissions, adhering to the principle of least privilege can be challenging due to the dynamic nature of architecture. Administrators may opt for more permissive permissions while implementing compensating security controls to minimize the blast radius.

See also

- DataOps – Aspect of data analytics
- DevOps toolchain – Tools for software development
- Infrastructure as code – Data center management method
- Lean software development – Use of lean manufacturing principles in software development
- List of build automation software
- Site reliability engineering – Use of software engineering practices for IT
- Value stream – Principle in economics
- Twelve-Factor App methodology – Software methodology

Notes

a. a portmanteau of **dev**elopment **op**erations
b. Dyck et al. (2015) "To our knowledge, there is no uniform definition for the terms release engineering and DevOps. As a consequence, many people use their own definitions or rely on others, which results in confusion about those terms."[4]
c. Jabbari et al. (2016) "The research results of this study showed the need for a definition as individual studies do not consistently define DevOps."[5]
d. Erich et al. (2017) "We noticed that there are various gaps in the study of DevOps: There is no consensus of what concepts DevOps covers, nor how DevOps is defined."[6]
e. Erich et al. (2017) "We discovered that there exists little agreement about the characteristics of DevOps in the academic literature."[6]

References

1. Courtemanche, Meredith; Mell, Emily; Gills, Alexander S. "What Is DevOps? The Ultimate Guide" (https://www.techtarget.com/searchitoperations/definition/DevOps). *TechTarget*.

Retrieved 2023-01-22.
2. *Building Evolutionary Architectures: Automated Software Governance*. ISBN 978-1-4920-9754-9.
3. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media. 2020. ISBN 978-1-4920-4345-4.
4. Dyck, Andrej; Penners, Ralf; Lichter, Horst (2015-05-19). "Towards Definitions for Release Engineering and DevOps". *2015 IEEE/ACM 3rd International Workshop on Release Engineering*. IEEE. p. 3. doi:10.1109/RELENG.2015.10 (https://doi.org/10.1109%2FRELENG.2015.10). ISBN 978-1-4673-7070-7. S2CID 4659735 (https://api.semanticscholar.org/CorpusID:4659735).
5. Jabbari, Ramtin; bin Ali, Nauman; Petersen, Kai; Tanveer, Binish (May 2016). "What is DevOps?: A Systematic Mapping Study on Definitions and Practices". *Proceedings of the 2016 Scientific Workshop*. Association for Computing Machinery.
6. Erich, F.M.A.; Amrit, C.; Daneva, M. (June 2017). "A Qualitative Study of DevOps Usage in Practice" (https://ris.utwente.nl/ws/files/19208022/Erich_et_al_2017_Journal_of_Software_Evolution_and_Process.pdf) (PDF). *Journal of Software: Evolution and Process*. **29** (6) e1885. doi:10.1002/smr.1885 (https://doi.org/10.1002%2Fsmr.1885). S2CID 35914007 (https://api.semanticscholar.org/CorpusID:35914007).
7. Bass, Len; Weber, Ingo; Zhu, Liming (2015). *DevOps: A Software Architect's Perspective*. Addison-Wesley. ISBN 978-0-13-404984-7.
8. Muñoz, Mirna; Negrete Rodríguez, Mario (2021-04-01). "A guidance to implement or reinforce a DevOps approach in organizations: A case study" (https://onlinelibrary.wiley.com/journal/20477481). *Journal of Software: Evolution and Process*. **36** (3). Wiley. doi:10.1002/smr.2342 (https://doi.org/10.1002%2Fsmr.2342).
9. Chapman, M., Gatti, N: A model of a service life cycle, Proceedings of TINA '93, pp. I-205–I-215, Sep., 1993.
10. Mezak, Steve (25 January 2018). "The Origins of DevOps: What's in a Name?" (https://devops.com/the-origins-of-devops-whats-in-a-name/). devops.com. Retrieved 6 May 2019.
11. Debois, Patrick (9 October 2008). "Agile 2008 Toronto" (http://www.jedi.be/blog/2008/10/09/agile-2008-toronto-agile-infrastructure-and-operations-presentation/). Just Enough Documented Information. Retrieved 12 March 2015.
12. Debois, Patrick. "DevOps Days" (http://www.devopsdays.org/). DevOps Days. Retrieved 31 March 2011.
13. Alana Brown; Nicole Forsgren; Jez Humble; Nigel Kersten; Gene Kim (2016). "2016 State of DevOps Report" (https://dora.dev/research/2017-and-earlier/2016-state-of-devops-report.pdf) (PDF). Puppet Labs, DORA (DevOps Research. Retrieved 2024-04-24.
14. "Puppet - Alanna Brown" (https://puppet.com/people/alanna-brown). Puppet Labs. Retrieved 2019-04-27.
15. Nicole Forsgren; Gene Kim; Nigel Kersten; Jez Humble (2014). "2014 State of DevOps Report" (https://dora.dev/research/2017-and-earlier/2014-state-of-devops-report.pdf) (PDF). Puppet Labs, IT Revolution Press and ThoughtWorks. Retrieved 2024-04-24.
16. "2015 State of DevOps Report" (https://dora.dev/research/2017-and-earlier/2015-state-of-devops-report.pdf) (PDF). Puppet Labs, Pwc, IT Revolution Press. 2015. Retrieved 2024-04-24.
17. "More Agile Testing" (https://agiletester.ca/wp-content/uploads/sites/26/2014/09/TOC.pdf) (PDF). October 2014. Retrieved 2019-05-06.
18. Crispin, Lisa; Gregory, Janet (October 2014). *More Agile Testing* (https://www.oreilly.com/library/view/more-agile-testing/9780133749571/). Addison-Wesley. ISBN 978-0-13-374957-1. Retrieved 2019-05-06.

19. Turner, Graham (20 November 2023). "Report: Software Engineers Face Backlash for Reporting Wrongdoing" (https://www.digit.fyi/report-software-engineers-facing-retaliation-for-reporting-wrongdoing/). *DIGIT*. Retrieved 5 January 2024.
20. Saran, Cliff. "Software engineers worry about speaking out - Computer Weekly" (https://www.computerweekly.com/news/366560292/Software-engineers-worry-about-speaking-out). *ComputerWeekly.com*. Retrieved 5 January 2024.
21. "75% of software engineers faced retaliation the last time they reported wrongdoing - ETHRWorldSEA" (https://hrsea.economictimes.indiatimes.com/news/workplace/75-of-software-engineers-faced-retaliation-the-last-time-they-reported-wrongdoing/105335733). *ETHRWorld.com*.
22. Cummins, Holly. "Holly Cummins on X" (https://twitter.com/holly_cummins/status/1448357917384744964). *X.com*. Retrieved 5 January 2024.
23. DeBellis, Derek; Lewis, Amanda; Villalba, Daniella; Farley, Dave. "2023 State of DevOps Report" (https://cloud.google.com/devops/state-of-devops). Google Cloud DevOps Research and Assessment. Retrieved 2024-04-24.
24. DeBellis, Derek; Harvey, Nathan. "2023 State of DevOps Report: Culture is everything" (https://cloud.google.com/blog/products/devops-sre/announcing-the-2023-state-of-devops-report). *Google Cloud Blog*. Retrieved 2024-04-24.
25. Forsgren, Nicole. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. ISBN 978-1-942788-33-1.
26. "DORA Accelerate State of DevOps 2021". *DevOps Research and Assessment* (2021). 2021.
27. Klein, Brandon Thorin (2021-05-01). "The DevOps: A Concise Understanding to the DevOps Philosophy and Science" (https://www.osti.gov/biblio/1785164/). *Osti.gov*. doi:10.2172/1785164 (https://doi.org/10.2172%2F1785164). OSTI 1785164 (https://www.osti.gov/biblio/1785164). S2CID 236606284 (https://api.semanticscholar.org/CorpusID:236606284).
28. "The History and Evolution of DevOps | Tom Geraghty" (https://tomgeraghty.co.uk/index.php/the-history-and-evolution-of-devops/). 5 July 2020. Retrieved 2020-11-29.
29. "What is platform engineering?" (https://learn.microsoft.com/en-us/platform-engineering/what-is-platform-engineering). *Microsoft Learn*. Retrieved 2025-04-01.
30. "What is platform engineering?" (https://www.redhat.com/en/topics/devops/platform-engineering). *Red Hat*. Retrieved 2025-04-01.
31. "Principles behind the Agile Manifesto" (https://agilemanifesto.org/principles.html). *agilemanifesto.org*. Retrieved 2020-12-06.
32. Castellanos, Camilo; Correal, Dario (15 September 2018). "Executing Architectural Models for Big Data Analytics". *Software Architecture*. Lecture Notes in Computer Science. Vol. 11048. pp. 364–371. doi:10.1007/978-3-030-00761-4_24 (https://doi.org/10.1007%2F978-3-030-00761-4_24). ISBN 978-3-030-00760-7.
33. Humble, Jez; Farley, David (2011). *Continuous Delivery: reliable software releases through build, test, and deployment automation*. Pearson Education Inc. ISBN 978-0-321-60191-9.
34. Chen, Lianping (2015). "Continuous Delivery: Huge Benefits, but Challenges Too". *IEEE Software*. **32** (2): 50–54. doi:10.1109/MS.2015.27 (https://doi.org/10.1109%2FMS.2015.27). S2CID 1241241 (https://api.semanticscholar.org/CorpusID:1241241).
35. Fluri, Jasmin; Fornari, Fabrizio; Pustulka, Elzbieta (2024). "On the importance of CI/CD practices for database applications" (https://irf.fhnw.ch/server/api/core/bitstreams/5f365e90-bc6a-49e0-8385-0be7e252d78a/content). *Journal of Software: Evolution and Process*. doi:10.1002/smr.2720 (https://doi.org/10.1002%2Fsmr.2720). Retrieved 2025-08-24.

36. Shahin, Mojtaba; Ali Babar, Muhammad; Zhu, Liming (2017). "Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices" (https://www.researchgate.net/publication/315381994_Continuous_Integration_Delivery_and_Deployment_A_Systematic_Review_on_Approaches_Tools_Challenges_and_Practices). *IEEE Access*. **5**: 3909–3943. arXiv:1703.07019 (https://arxiv.org/abs/1703.07019). doi:10.1109/ACCESS.2017.2685629 (https://doi.org/10.1109%2FACCESS.2017.2685629). Retrieved 2025-08-24.
37. "Database Change Management" (https://dora.dev/capabilities/database-change-management/). *DORA*. Retrieved 2025-08-24.
38. "A Guide to Database DevOps" (https://www.liquibase.com/resources/guides/database-devops#:~:text=Database%20DevOps%20extends%20DevOps%20principles%2C,more%20consistent%20and%20efficient%20lifecycle). *Liquibase*. Retrieved 2025-08-24.
39. Tak, Rohin; Modi, Jhalak (2018). *Mobile DevOps: Deliver continuous integration and deployment within your mobile applications*. Packt Publishing. pp. 12–18. ISBN 978-1-78829-624-3.
40. Beyer, Betsy; Jones, Chris; Petoff, Jennifer; Murphy, Niall Richard (April 2016). *Site Reliability Engineering*. O'Reilly Media. ISBN 978-1-4919-2909-4.
41. Dave Harrison (9 Oct 2018). "Interview with Betsy Beyer, Stephen Thorne of Google" (https://driftboatdave.com/2018/10/09/interview-with-betsy-beyer-stephen-thorne-of-google/). Retrieved 24 July 2024.
42. Analyzing the DNA of DevOps (https://opensource.com/article/18/11/analyzing-devops), Brent Aaron Reed, Willy Schaub, 2018-11-14.
43. Gene Kim; Patrick Debois; John Willis; Jezz Humble (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*.
44. "CWE Top 25 Most Dangerous Software Weaknesses" (https://cwe.mitre.org/top25/). Retrieved September 21, 2025.
45. "OWASP TOP10" (https://owasp.org/Top10/). Archived (https://web.archive.org/web/20230608171837/https://owasp.org/Top10/) from the original on June 8, 2023. Retrieved June 8, 2023.
46. Wilson, Glenn (December 2020). *'DevSecOps: A leader's guide to producing secure software without compromising flow, feedback and continuous improvement'*. Rethink Press. ISBN 978-1-78133-502-4.
47. Emerging Technology Analysis: DevOps a Culture Shift, Not a Technology (Report). Gartner.
48. Loukides, Mike (7 June 2012). "What is DevOps?" (http://radar.oreilly.com/2012/06/what-is-devops.html). O'Reilly Media.
49. Teja Yarlagadda, Ravi (9 March 2021). "DevOps and Its Practices". SSRN 3798877 (https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3798877).
50. Morisio, Maurizio (16 April 2021). *DevOps: development of a toolchain in the banking domain* (https://webthesis.biblio.polito.it/18120/). *Politecnico di Torino* (laurea thesis). Retrieved 16 August 2021.
51. "What is GitOps?" (https://www.redhat.com/en/topics/devops/what-is-gitops). *www.redhat.com*. Retrieved 2023-03-30.
52. *Serverless Architectures on AWS*. Manning. 29 March 2022. ISBN 978-1-61729-542-3.
53. *Pipeline as Code Continuous Delivery with Jenkins, Kubernetes, and Terraform*. Manning. 23 November 2021. ISBN 978-1-63835-037-8.
54. Humble, Jez; Farley, David (27 July 2010). *Continuous Delivery Reliable Software Releases Through Build, Test, and Deployment Automation*. Pearson Education. ISBN 978-0-321-67022-9.

Further reading

- Davis, Jennifer; Daniels, Ryn (2016-05-30). *Effective DevOps: building a culture of collaboration, affinity, and tooling at scale*. Sebastopol, CA: O'Reilly. ISBN 978-1-4919-2643-7. OCLC 951434424 (https://search.worldcat.org/oclc/951434424).
- Kim, Gene; Debois, Patrick; Willis, John; Humble, Jez; Allspaw, John (2015-10-07). *The DevOps handbook: how to create world-class agility, reliability, and security in technology organizations* (First ed.). Portland, OR. ISBN 978-1-942788-00-3. OCLC 907166314 (https://search.worldcat.org/oclc/907166314).
- Forsgren, Nicole; Humble, Jez; Kim, Gene (27 March 2018). *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations* (First ed.). IT Revolution Press. ISBN 978-1-942788-33-1.

Retrieved from "https://en.wikipedia.org/w/index.php?title=DevOps&oldid=1338158639"